

- SkinAI - Sistem Deteksi Penyakit Kulit Wajah Berbasis Kecerdasan Buatan
 -  Informasi Project
 -  Tentang Project
 - Latar Belakang
 - Tujuan Project
 - Rumusan Masalah
 -  Fitur Utama
 - 1. Klasifikasi Kondisi Kulit (Skin Classification)
 - 2. Rekomendasi Produk Perawatan
 - 3. Sistem Autentikasi Pengguna
 - 4. Riwayat Prediksi (Prediction History)
 - 5. Profil Pengguna
 - 6. Manajemen Artikel (Admin)
 - 7. Health Check API
 -  Arsitektur Sistem
 - Diagram Arsitektur
 - Alur Kerja Sistem (System Workflow)
 -  Teknologi yang Digunakan
 - Frontend Stack
 - AI/ML Stack
 - Backend Stack
 - Authentication & Security
 - Development Tools
 -  Struktur Project
 -  Struktur Database
 - Entity Relationship Diagram (ERD)
 - Tabel Database
 - 1. app_users
 - 2. predictions
 - 3. articles
 - 4. admin_users
 - Row Level Security (RLS) Policies
 -  Model Machine Learning
 - Spesifikasi Model
 - Kelas Klasifikasi
 - Pipeline Preprocessing
 - Proses Inferensi

-  [API Endpoints](#)
 - [1. POST /api/predictions](#)
 - [2. GET /api/articles](#)
 - [3. POST /api/articles \(Admin Only\)](#)
 - [4. GET /api/health](#)
-  [Instalasi dan Deployment](#)
 - [Prasyarat](#)
 - [Langkah Instalasi](#)
 - [Deployment ke Vercel](#)
-  [Penggunaan Aplikasi](#)
 - [Alur Penggunaan Umum](#)
-  [Keamanan dan Privasi](#)
 - [Prinsip Privacy-First](#)
 - [Protected Routes](#)
-  [Referensi](#)
 - [Teknologi & Framework](#)
 - [Deep Learning](#)
 - [Dataset Produk Skincare](#)
-  [Kontributor](#)
-  [Lisensi](#)
-  [Kontak](#)

SkinAI - Sistem Deteksi Penyakit Kulit Wajah Berbasis Kecerdasan Buatan



Aplikasi Web untuk Klasifikasi Kondisi Kulit Wajah menggunakan Deep Learning dan ONNX Runtime

[Tentang](#) • [Fitur](#) • [Arsitektur](#) • [Teknologi](#) • [Instalasi](#) • [Penggunaan](#)



Informasi Project

|-----|-----|

| **Nama Aplikasi** | SkinAI |

| **Versi** | 0.1.0 |

| **Jenis** | Aplikasi Web Progressive |

| **Domain** | Kesehatan Kulit / Dermatologi |

| **Metode AI** | Deep Learning - Convolutional Neural Network |

| **Runtime** | Client-side (Browser-based Inference) |



Tentang Project

Latar Belakang

Kesehatan kulit wajah merupakan salah satu aspek penting dalam kesehatan secara keseluruhan. Banyak orang mengalami berbagai masalah kulit seperti jerawat (*acne*), komedo (*blackheads*), flek hitam (*dark spots*), kulit berminyak (*oily skin*), dan kerutan (*wrinkles*). Namun, tidak semua orang memiliki akses ke dermatolog atau dokter kulit untuk mendapatkan diagnosis yang tepat.

SkinAI hadir sebagai solusi teknologi yang memanfaatkan kecerdasan buatan untuk membantu pengguna mengidentifikasi kondisi kulit wajah mereka secara mandiri. Aplikasi ini menggunakan model *deep learning* yang telah dilatih untuk mengklasifikasikan berbagai kondisi kulit berdasarkan gambar yang diunggah pengguna.

Tujuan Project

- Aksesibilitas:** Memberikan akses diagnosis awal kondisi kulit kepada masyarakat luas tanpa perlu kunjungan ke dokter spesialis.
- Privasi:** Menjamin privasi pengguna dengan menjalankan inferensi model sepenuhnya di browser (*client-side*), sehingga gambar tidak dikirim ke server.

- 3.**Edukasi:** Memberikan rekomendasi produk perawatan kulit berdasarkan hasil diagnosis untuk membantu pengguna merawat kulit mereka.
- 4.**Kemudahan Penggunaan:** Menyediakan antarmuka yang intuitif dan responsif yang dapat diakses dari berbagai perangkat.

Rumusan Masalah

1. Bagaimana mengimplementasikan model *deep learning* untuk klasifikasi kondisi kulit wajah pada platform web?
2. Bagaimana menjalankan inferensi model AI di sisi klien (*client-side*) untuk menjaga privasi pengguna?
3. Bagaimana mengintegrasikan sistem rekomendasi produk berdasarkan hasil klasifikasi?
4. Bagaimana membangun sistem autentikasi dan manajemen riwayat prediksi pengguna?

Fitur Utama

1. Klasifikasi Kondisi Kulit (Skin Classification)

Fitur utama aplikasi yang menggunakan model ONNX untuk mengklasifikasikan kondisi kulit wajah ke dalam 6 kategori:

No	Kategori	Deskripsi
----	-----	-----
1	Acne	Jerawat - kondisi peradangan pada kulit akibat penyumbatan pori-pori
2	Blackheads	Komedo hitam - pori-pori tersumbat yang teroksidasi
3	Dark Spots	Flek hitam / hiperpigmentasi pada kulit
4	Normal Skin	Kulit normal tanpa masalah signifikan
5	Oily Skin	Kulit berminyak dengan produksi sebum berlebih

Proses Klasifikasi:

1. Pengguna mengunggah gambar atau mengambil foto menggunakan kamera
2. Gambar diproses (*preprocessing*) dengan teknik center-crop dan normalisasi
3. Model ONNX melakukan inferensi untuk menghasilkan skor probabilitas
4. Sistem menampilkan hasil klasifikasi beserta tingkat keyakinan (*confidence score*)

2. Rekomendasi Produk Perawatan

Berdasarkan hasil klasifikasi, sistem memberikan rekomendasi produk skincare yang sesuai:

- Database Produk:** 100+ produk dari berbagai brand lokal dan internasional
- Informasi Produk:** Nama produk, brand, harga, dan link pembelian
- Kategorisasi:** Produk dikategorikan berdasarkan jenis masalah kulit

3. Sistem Autentikasi Pengguna

- Registrasi:** Pendaftaran akun baru dengan email dan password
- Login:** Masuk ke akun dengan kredensial yang terdaftar
- Session Management:** Pengelolaan sesi menggunakan Supabase Auth dengan cookie-based authentication

4. Riwayat Prediksi (Prediction History)

- Menyimpan semua hasil prediksi pengguna yang terautentikasi
- Menampilkan label klasifikasi, confidence score, sumber gambar (upload/kamera), dan timestamp
- Memungkinkan pengguna melacak kondisi kulit dari waktu ke waktu

5. Profil Pengguna

- Menampilkan informasi akun pengguna
- Ringkasan statistik total prediksi
- Akses cepat ke fitur-fitur utama

6. Manajemen Artikel (Admin)

- Admin dapat mempublikasikan artikel kesehatan kulit
- Artikel ditampilkan di halaman beranda dan halaman artikel
- Fitur CRUD untuk pengelolaan konten

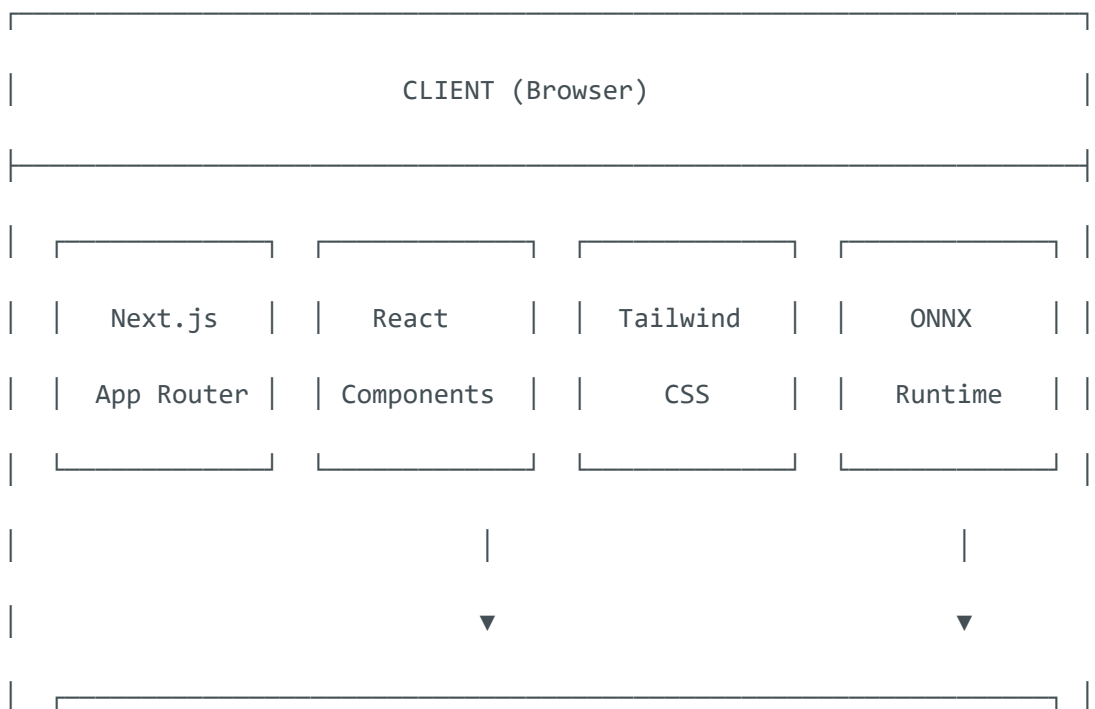
7. Health Check API

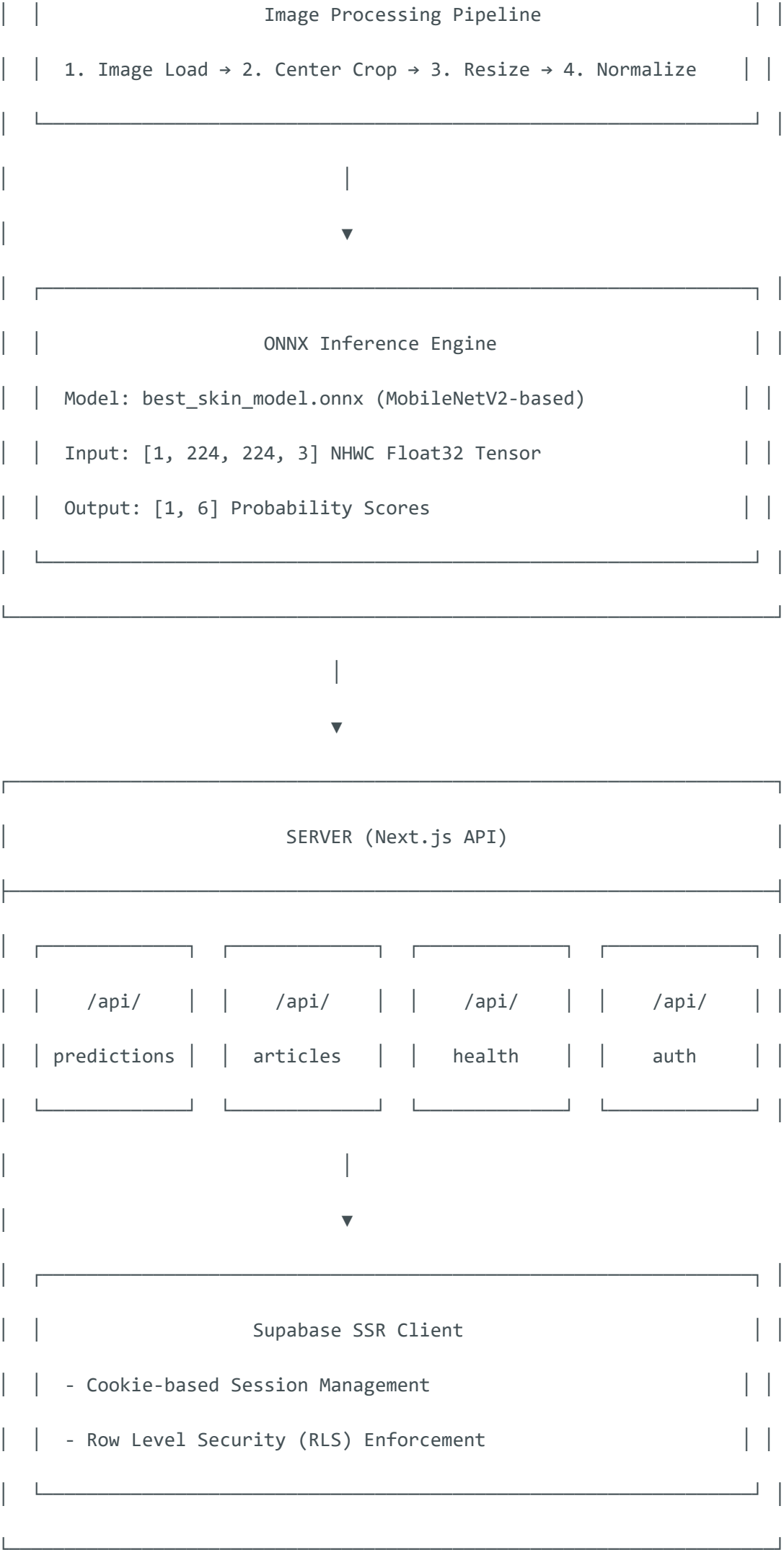
- Endpoint diagnostik untuk memverifikasi konektivitas Supabase
- Validasi environment variables
- Status sesi pengguna

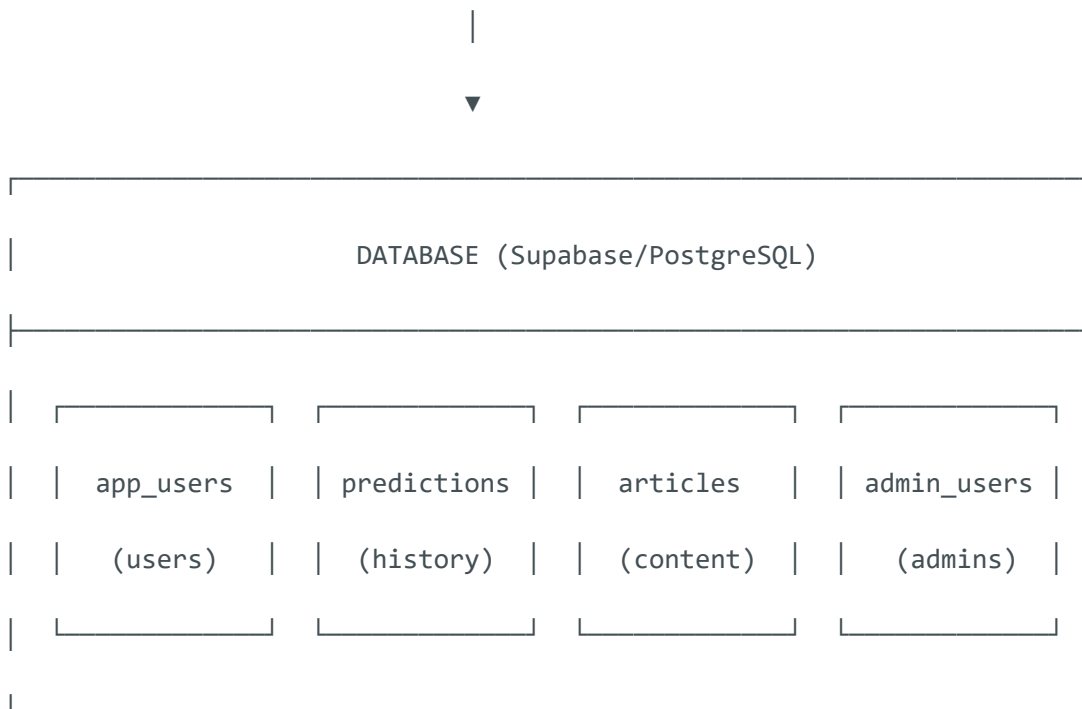


Arsitektur Sistem

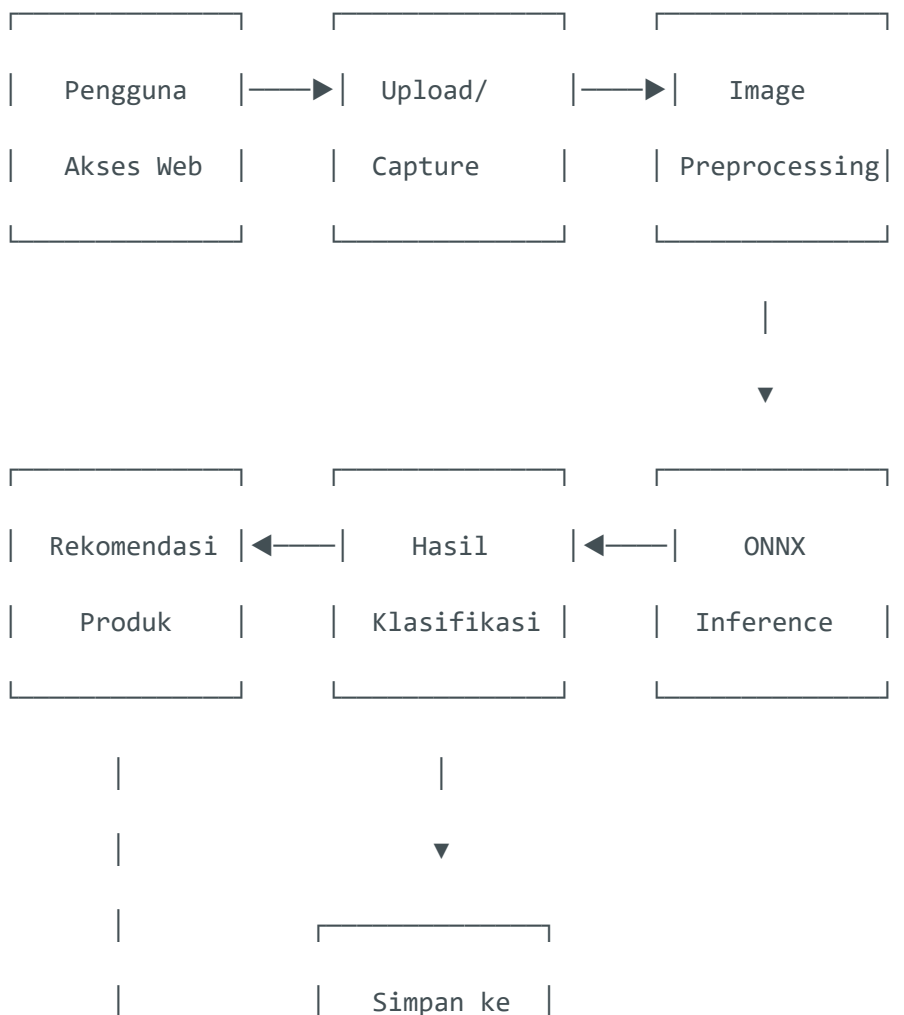
Diagram Arsitektur

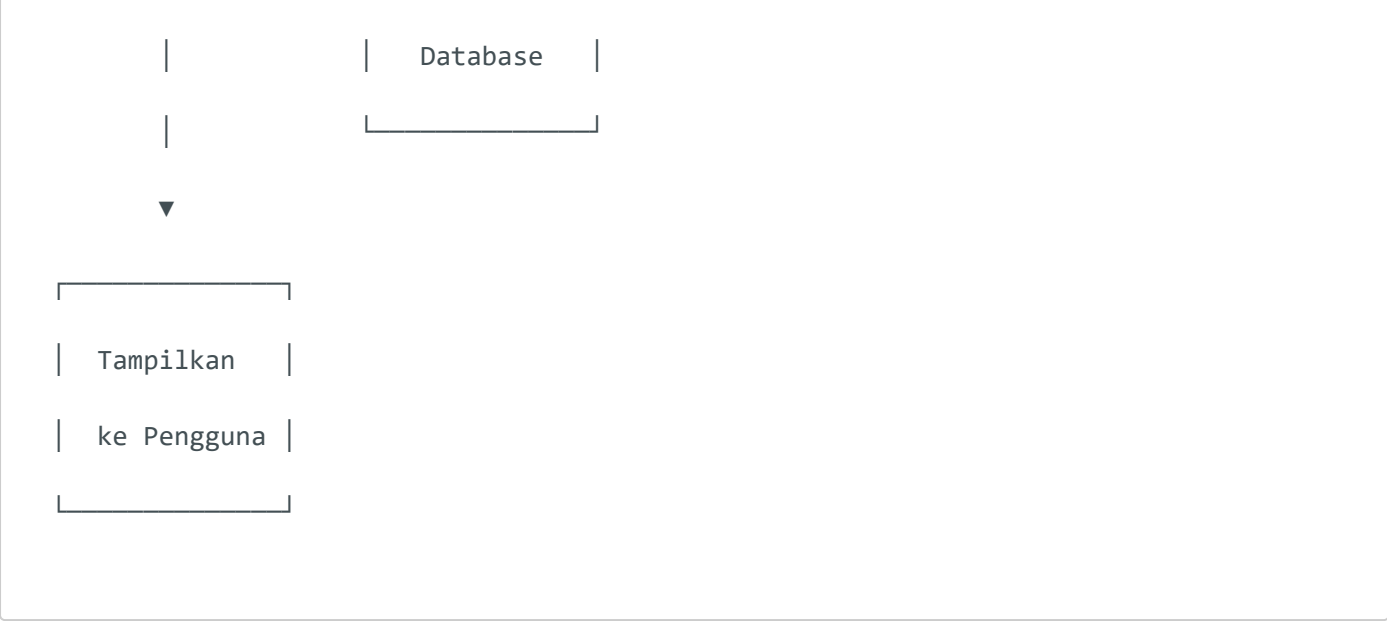






Alur Kerja Sistem (System Workflow)





Teknologi yang Digunakan

Frontend Stack

Teknologi	Versi	Fungsi
Next.js	14.2.x	Framework React dengan App Router untuk SSR/SSG
React	18.3.x	Library UI untuk membangun komponen interaktif
TypeScript	5.6.x	Superset JavaScript dengan static typing
Tailwind CSS	4.1.x	Utility-first CSS framework untuk styling responsif

AI/ML Stack

Teknologi	Versi	Fungsi
ONNX Runtime Web	1.23.x	Runtime untuk menjalankan model ONNX di browser
ONNX Model	-	Model deep learning dalam format Open Neural Network Exchange

Backend Stack

| Teknologi | Versi | Fungsi |

|-----|-----|-----|

| **Next.js API Routes** | 14.2.x | Serverless API endpoints |

| **Supabase** | - | Backend-as-a-Service (BaaS) |

| **@supabase/ssr** | - | Server-side rendering support untuk Supabase |

| **PostgreSQL** | - | Database relasional (via Supabase) |

Authentication & Security

| Teknologi | Fungsi |

|-----|-----|

| **Supabase Auth** | Autentikasi pengguna dengan email/password |

| **Cookie-based Sessions** | Manajemen sesi yang aman |

| **Row Level Security (RLS)** | Keamanan data di level database |

| **Middleware Protection** | Proteksi route untuk halaman terautentikasi |

Development Tools

| Tool | Fungsi |

|-----|-----|

| **ESLint** | Linting dan code quality |

| **Prettier** | Code formatting |

| **PostCSS** | CSS processing |

| **Autoprefixer** | Vendor prefix automation |



Struktur Project

SkinAI/

```
├─ app/                                # Next.js App Router
|   ├─ (protected)/                    # Route group untuk halaman terproteksi
|   |   ├─ admin/                      # Halaman admin
|   |   |   └─ articles/               # Manajemen artikel
|   |   └─ history/                   # Riwayat prediksi
|   |       └─ profile/                # Profil pengguna
|   └─ about/                         # Halaman tentang
|   └─ api/                           # API Routes
|       ├─ articles/                 # CRUD artikel
|       |   └─ route.ts               # GET/POST articles
|       |       └─ sanitize/          # URL sanitization
|       └─ auth/                     # Autentikasi
|           ├─ health/               # Health check endpoint
|           └─ predictions/          # Simpan prediksi
|   └─ articles/                     # Daftar artikel publik
|   └─ login/                        # Halaman login
|   └─ predict/                      # Halaman prediksi utama
|   └─ register/                    # Halaman registrasi
|   └─ globals.css                   # Global styles
|   └─ layout.tsx                    # Root layout
|       └─ page.tsx                  # Homepage
├─ components/                       # React Components
|   └─ AuthButtons.tsx               # Komponen autentikasi
|   └─ Footer.tsx                   # Footer
|   └─ Navbar.tsx                   # Navigation bar
```

```

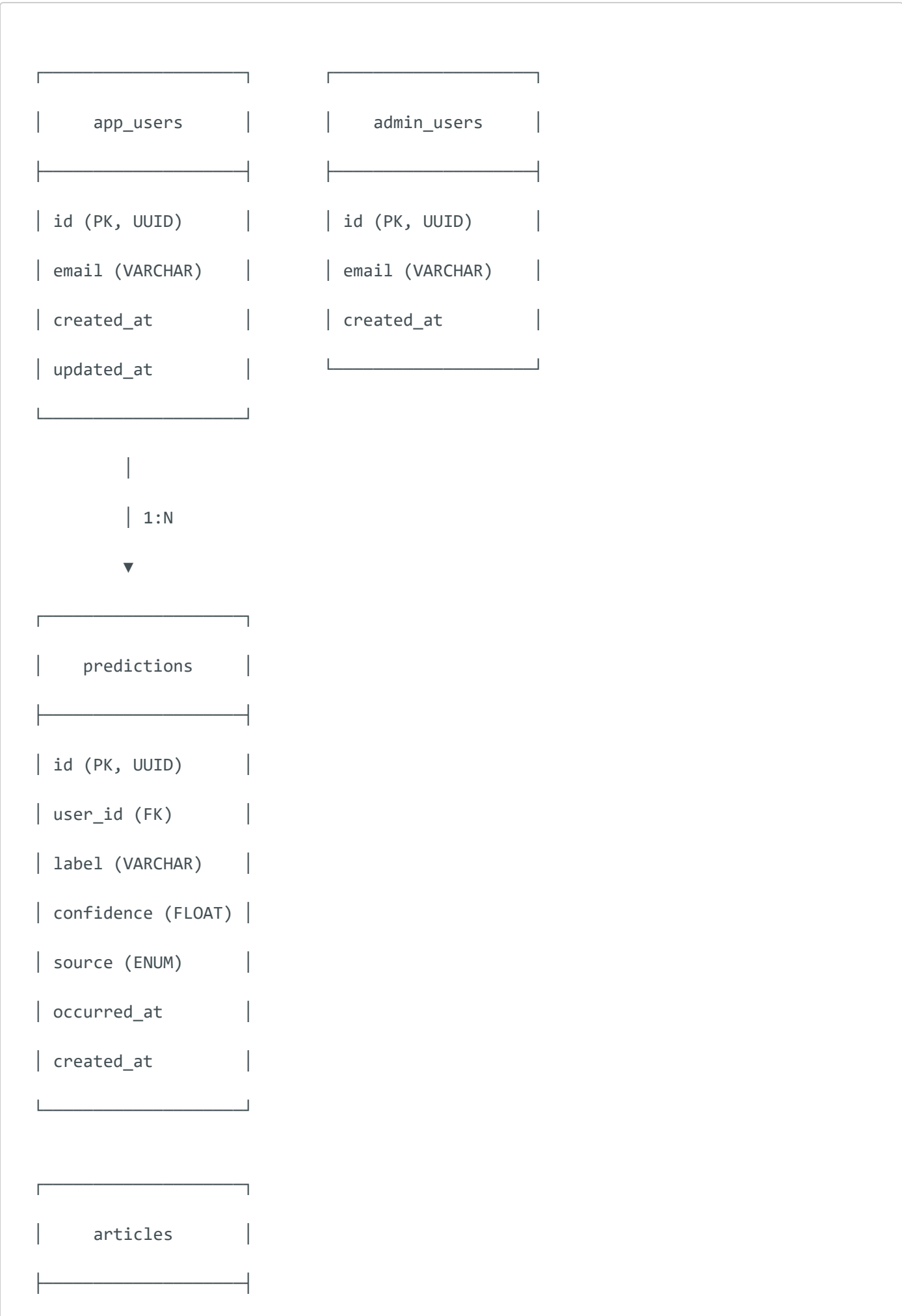
|   └─ ThemeToggle.tsx          # Toggle tema (dark/light)
└─ lib/                          # Library utilities
|   └─ modelPrefetch.ts        # Prefetch model ONNX
|   └─ supabaseClient.ts       # Supabase browser client
|   └─ supabaseServer.ts       # Supabase server client
└─ public/                      # Static assets
|   └─ brandLogo.png           # Logo aplikasi
|   └─ brandProfile-large.png  # Hero image
|   └─ model/                  # Model ONNX
|       └─ best_skin_model.onnx # Trained model
└─ skincare_product/           # Dataset produk
    └─ gambar_produk/          # Gambar produk (100+ images)
        └─ treatment.csv       # Data produk (CSV)
            └─ treatment.json   # Data produk (JSON)
└─ types/                      # TypeScript type definitions
    └─ index.ts                # Type exports
└─ utils/                      # Utility functions
    └─ supabase/               # Supabase helpers
        └─ server.ts           # Server-side client
└─ middleware.ts               # Next.js middleware (route protection)
└─ next.config.js              # Next.js configuration
└─ tailwind.config.js          # Tailwind CSS configuration
└─ tsconfig.json               # TypeScript configuration
└─ package.json                # Dependencies & scripts

```



Struktur Database

Entity Relationship Diagram (ERD)



id (PK, UUID)	
title (VARCHAR)	
summary (TEXT)	
content_url (URL)	
cover_url (URL)	
source (ENUM)	
created_at	

Tabel Database

1. app_users

Menyimpan data pengguna aplikasi.

Kolom	Tipe	Deskripsi
-------	------	-----------

-----	-----	-----
-------	-------	-------

id	UUID	Primary key, referensi ke auth.users
-----------	------	--------------------------------------

email	VARCHAR	Email pengguna
--------------	---------	----------------

created_at	TIMESTAMPTZ	Waktu pembuatan akun
-------------------	-------------	----------------------

updated_at	TIMESTAMPTZ	Waktu pembaruan terakhir
-------------------	-------------	--------------------------

2. predictions

Menyimpan riwayat prediksi pengguna.

Kolom	Tipe	Deskripsi
-------	------	-----------

-----	-----	-----
-------	-------	-------

id	UUID	Primary key
-----------	------	-------------

user_id	UUID	Foreign key ke app_users
----------------	------	--------------------------

| **label** | VARCHAR | Hasil klasifikasi (Acne, Blackheads, dll) |

| **confidence** | FLOAT | Tingkat keyakinan (0-1) |

| **source** | ENUM | Sumber gambar ('upload' atau 'camera') |

| **occurred_at** | TIMESTAMPTZ | Waktu prediksi dilakukan |

| **created_at** | TIMESTAMPTZ | Waktu record dibuat |

3. **articles**

Menyimpan artikel kesehatan kulit.

| Kolom | Tipe | Deskripsi |

|-----|-----|-----|

| **id** | UUID | Primary key |

| **title** | VARCHAR | Judul artikel |

| **summary** | TEXT | Ringkasan artikel |

| **content_url** | URL | Link ke konten lengkap |

| **cover_url** | URL | Link gambar sampul |

| **source** | ENUM | Sumber ('koran' atau 'bacaan') |

| **created_at** | TIMESTAMPTZ | Waktu publikasi |

4. **admin_users**

Menyimpan daftar administrator.

| Kolom | Tipe | Deskripsi |

|-----|-----|-----|

| **id** | UUID | Primary key, referensi ke auth.users |

| **email** | VARCHAR | Email admin |

| **created_at** | TIMESTAMPTZ | Waktu ditambahkan sebagai admin |

Row Level Security (RLS) Policies

```
-- app_users: Users can only read/update their own data

CREATEPOLICY"Users can view own profile"ON app_users

    FORSELECTUSING (auth.uid() = id);

CREATEPOLICY"Users can update own profile"ON app_users

    FORUPDATEUSING (auth.uid() = id);

-- predictions: Users can only access their own predictions

CREATEPOLICY"Users can view own predictions"ON predictions

    FORSELECTUSING (auth.uid() = user_id);

CREATEPOLICY"Users can insert own predictions"ON predictions

    FORINSERTWITHCHECK (auth.uid() = user_id);

-- articles: Public read, admin-only write

CREATEPOLICY"Anyone can view articles"ON articles

    FORSELECTUSING (true);

CREATEPOLICY"Admins can insert articles"ON articles

    FORINSERTWITHCHECK (

        EXISTS (SELECT1FROM admin_users WHERE id =auth.uid())

    );
```



Model Machine Learning

Spesifikasi Model

| Aspek | Detail |

|-----|-----|

| **Format** | ONNX (Open Neural Network Exchange) |

| **Arsitektur Base** | MobileNetV2 (Transfer Learning) |

| **Input Shape** | [1, 224, 224, 3] (NHWC format) |

| **Output Shape** | [1, 6] (6 class probabilities) |

| **File Size** | ~14 MB |

| **Normalization** | [-1, 1] range (pixel / 127.5 - 1) |

Kelas Klasifikasi

```
const categories = [  
  "Acne",          // Index 0  
  "Blackheads",    // Index 1  
  "Dark Spots",    // Index 2  
  "Normal Skin",   // Index 3  
  "Oily Skin",     // Index 4  
  "Wrinkles",      // Index 5  
]
```

Pipeline Preprocessing

```
function preprocessImage(img, size=224) {  
  // 1. Create canvas for image manipulation  
  const canvas = document.createElement("canvas")
```

```
canvas.width = size

canvas.height = size

const ctx = canvas.getContext("2d")

// 2. Center-crop to maintain aspect ratio

const sw = img.naturalWidth

const sh = img.naturalHeight

const ar = sw / sh

let sx = 0, sy = 0, sWidth = sw, sHeight = sh

if (ar > 1) {

    sWidth = sh

    sx = (sw - sWidth) / 2

} else {

    sHeight = sw

    sy = (sh - sHeight) / 2

}

// 3. Draw and resize

ctx.drawImage(img, sx, sy, sWidth, sHeight, 0, 0, size, size)

// 4. Extract pixel data

const imageData = ctx.getImageData(0, 0, size, size)

// 5. Normalize to [-1, 1] range (MobileNetV2 style)

const arr = new Float32Array(size * size * 3)

for (let i = 0; i < size * size; i++) {

    arr[i * 3 + 0] = imageData.data[i * 4 + 0] / 127.5 - 1 // R

    arr[i * 3 + 1] = imageData.data[i * 4 + 1] / 127.5 - 1 // G

    arr[i * 3 + 2] = imageData.data[i * 4 + 2] / 127.5 - 1 // B
```

```

}

// 6. Return as ONNX Tensor

return new ort.Tensor("float32", arr, [1, size, size, 3])
}

```

Proses Inferensi

```

async function runInference(imageTensor) {

  // 1. Get input name from model

  const inputName = session.inputNames[0]

  // 2. Prepare feeds

  const feeds = { [inputName]: imageTensor }

  // 3. Run inference

  const output = await session.run(feeds)

  // 4. Get output scores

  const outputName = session.outputNames[0]

  const scores = output[outputName].data // Float32Array[6]

  // 5. Find best prediction (argmax)

  let bestIdx = 0

  for (let i = 1; i < scores.length; i++) {

    if (scores[i] > scores[bestIdx]) bestIdx = i

  }
}

```

```
return {  
  label: categories[bestIdx],  
  confidence: scores[bestIdx]  
}  
}
```

API Endpoints

1. POST `/api/predictions`

Menyimpan hasil prediksi ke database.

Request:

```
{  
  "label": "Acne",  
  "confidence": 0.89,  
  "source": "upload",  
  "occurred_at": "2024-01-15T10:30:00.000Z"  
}
```

Response (Success):

```
{  
  "ok": true,  
  "data": {  
    "id": "uuid-xxx",
```

```
"user_id": "user-uuid",

"label": "Acne",

"confidence": 0.89,

"source": "upload",

"occurred_at": "2024-01-15T10:30:00.000Z"

}

}
```

Response (Error - Not Authenticated):

```
{

  "error": "Not authenticated",

  "hint": "Login required before saving prediction."

}
```

2. GET `/api/articles`

Mengambil daftar artikel terbaru.

Response:

```
{

  "data": [

    {

      "id": "uuid-xxx",

      "title": "Tips Merawat Kulit Berjerawat",

      "summary": "Panduan lengkap perawatan kulit...",

      "cover_url": "https://example.com/image.jpg",
```

```
    "created_at": "2024-01-15T10:30:00.000Z",  
    "source": "bacaan"  
  }  
]  
}
```

3. POST `/api/articles` (Admin Only)

Membuat artikel baru.

Request:

```
{  
  "title": "Judul Artikel",  
  "summary": "Ringkasan artikel...",  
  "content_url": "https://example.com/article",  
  "cover_url": "https://example.com/cover.jpg",  
  "source": "koran"  
}
```

4. GET `/api/health`

Health check untuk diagnostik sistem.

Response:

```
{  
  "ok": true,
```

```
"envOk": true,  
  
"session": { "id": "user-uuid", "email": "user@email.com" },  
  
"userError": null,  
  
"canQuery": true,  
  
"queryError": null  
  
}
```



Instalasi dan Deployment

Prasyarat

-Node.js >= 18.x

-npm atau yarn

-Akun Supabase (untuk database dan authentication)

Langkah Instalasi

1.Clone Repository

```
git clone https://github.com/pael611/SkinAI.git  
  
cd SkinAI
```

2.Install Dependencies

```
npm install
```

3.Konfigurasi Environment Variables

Buat file `.env.local` dengan isi:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co  
  
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_DEFAULT_KEY=your-anon-key
```

4.Setup Database (Supabase)

Jalankan SQL berikut di Supabase SQL Editor:

```
-- Create app_users table  
  
CREATETABLEpublic.app_users (  
  
  id UUID PRIMARY KEYREFERENCESauth.users(id),  
  
  email TEXT,  
  
  created_at TIMESTAMPTZDEFAULTnow(),  
  
  updated_at TIMESTAMPTZDEFAULTnow()  
  
);  
  
  
-- Create predictions table  
  
CREATETABLEpublic.predictions (  
  
  id UUID PRIMARY KEYDEFAULT gen_random_uuid(),  
  
  user_id UUID REFERENCESpublic.app_users(id),  
  
  label TEXTNOT NULL,  
  
  confidence FLOATNOT NULL,  
  
  source TEXTCHECK (source IN ('upload', 'camera')),  
  
  occurred_at TIMESTAMPTZDEFAULTnow(),  
  
  created_at TIMESTAMPTZDEFAULTnow()  
  
);
```



```
-- Create articles table
```

```
CREATETABLEpublic.articles (  
  
  id UUID PRIMARY KEYDEFAULT gen_random_uuid(),  
  
  title TEXTNOT NULL,  
  
  summary TEXT,  
  
  content_url TEXT,  
  
  cover_url TEXT,  
  
  source TEXTCHECK (source IN ('koran', 'bacaan')),  
  
  created_at TIMESTAMPTZDEFAULTnow()  
  
);
```

```
-- Create admin_users table
```

```
CREATETABLEpublic.admin_users (  
  
  id UUID PRIMARY KEYREFERENCESauth.users(id),  
  
  email TEXT,  
  
  created_at TIMESTAMPTZDEFAULTnow()  
  
);
```

```
-- Enable RLS
```

```
ALTERTABLEpublic.app_usersENABLEROWLEVELSECURITY;  
  
ALTERTABLEpublic.predictionsENABLEROWLEVELSECURITY;  
  
ALTERTABLEpublic.articlesENABLEROWLEVELSECURITY;  
  
ALTERTABLEpublic.admin_usersENABLEROWLEVELSECURITY;
```

```
-- RLS Policies
```

```
CREATEPOLICY"Users can view own data"ON app_users FORSELECTUSING (auth.uid() =  
id);
```

```
CREATEPOLICY"Users can insert own data"ON app_users FORINSERTWITHCHECK  
(auth.uid() = id);
```

```
CREATEPOLICY"Users can update own data"ON app_users FORUPDATEUSING (auth.uid() = id);
```

```
CREATEPOLICY"Users can view own predictions"ON predictions FORSELECTUSING (auth.uid() = user_id);
```

```
CREATEPOLICY"Users can insert predictions"ON predictions FORINSERTWITHCHECK (auth.uid() = user_id);
```

```
CREATEPOLICY"Anyone can view articles"ON articles FORSELECTUSING (true);
```

```
CREATEPOLICY"Admins can manage articles"ON articles FOR ALL USING (  
  EXISTS (SELECT1FROM admin_users WHERE id =auth.uid())  
);
```

5.Jalankan Development Server

```
npm run dev
```

Aplikasi akan berjalan di <http://localhost:3000>

6.Build untuk Production

```
npm run build
```

```
npm run start
```

Deployment ke Vercel

1. Push code ke GitHub
2. Connect repository ke Vercel
3. Set environment variables di Vercel Dashboard
4. Deploy!



Penggunaan Aplikasi

Alur Penggunaan Umum

1. Akses Aplikasi

- Buka aplikasi melalui browser
- Homepage menampilkan deskripsi dan artikel terbaru

2. Registrasi/Login (Opsional)

- Klik "Daftar" untuk membuat akun baru
- Atau "Masuk" jika sudah memiliki akun
- Diperlukan untuk menyimpan riwayat prediksi

3. Melakukan Prediksi

- Navigasi ke halaman "Predict"
- Tunggu model ONNX selesai dimuat
- Upload gambar wajah atau gunakan kamera
- Klik tombol "Prediksi"

4. Melihat Hasil

- Hasil klasifikasi ditampilkan dengan confidence score
- Rekomendasi produk muncul sesuai kategori
- Hasil otomatis tersimpan jika pengguna login

5. Melihat Riwayat

- Akses halaman "History" untuk melihat prediksi sebelumnya
- Lihat tren kondisi kulit dari waktu ke waktu



Keamanan dan Privasi

Prinsip Privacy-First

- 1.**Client-Side Inference**: Semua proses analisis gambar dilakukan di browser pengguna. Gambar tidak dikirim ke server manapun.
- 2.**Cookie-Based Authentication**: Sesi pengguna dikelola melalui HTTP-only cookies yang aman.
- 3.**Row Level Security**: Database Supabase dikonfigurasi dengan RLS untuk memastikan pengguna hanya dapat mengakses data mereka sendiri.
- 4.**Middleware Protection**: Route sensitif dilindungi oleh middleware yang memvalidasi sesi sebelum mengizinkan akses.

Protected Routes

| Route | Proteksi |

|-----|-----|

| [/profile](#) | Memerlukan autentikasi |

| [/history](#) | Memerlukan autentikasi |

| [/admin/*](#) | Memerlukan autentikasi + status admin |



Referensi

Teknologi & Framework

- [Next.js Documentation](#)
- [React Documentation](#)
- [Tailwind CSS](#)
- [ONNX Runtime Web](#)
- [Supabase Documentation](#)

Deep Learning

- MobileNetV2: Sandler, M., et al. (2018). "MobileNetV2: Inverted Residuals and Linear Bottlenecks"
- Transfer Learning for Medical Images
- ONNX Format Specification

Dataset Produk Skincare

- Sociolla Indonesia
- Dataset lokal dengan 100+ produk skincare



Kontributor

Rafael Siregar | Developer & Researcher |



Lisensi

Project ini dikembangkan untuk keperluan akademik dan penelitian.



Kontak

Untuk pertanyaan atau kolaborasi, hubungi:

-**Email:** rafaelsiregar611@gmail.com

-**GitHub:** [@pael611](#)

SkinAI - Perawatan Kulit Berbasis Kecerdasan Buatan

Dibuat dengan ❤️ menggunakan Next.js, React, dan ONNX Runtime